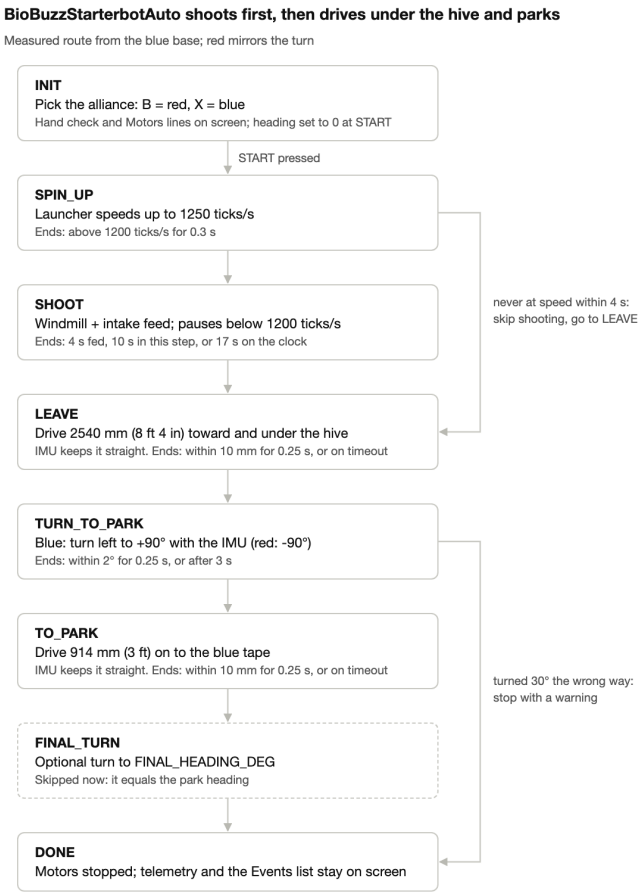
BioBuzzStarterbotAuto: How It Works and What It Needs

Nov 7, 2026 (telemetry updated Nov 12, 2026)

BioBuzzStarterbotAuto is the team's competition autonomous; on the Driver Station it is "BioBuzz StarterBot Auto", next to its teleop partner "BioBuzz StarterBot Teleop". It was called UltronAuto until Nov 12, 2026. Don't confuse it with goBILDA's sample `StarterBotAuto` (on the Driver Station: "goBILDA Sample Auto (reference)"), which is only for reference.

The auto starts facing the hive, shoots the 4 preloaded balls, drives 8 ft 4 in straight under the hive, turns left and drives 3 ft on to the parking spot (blue tape). The route is measured from the blue base. In a headless test model it finished in about 11.6 s of the 30 s autonomous.

What it does in a match



Each step ends only when its condition holds, so a timer or safety check is always running. In the headless test model, spin-up took about 1.4 s, shooting 4 s, leaving 3.7 s, the turn 0.9 s and parking 1.6 s. A missing launcher or intake skips from INIT straight to LEAVE.

How the code is organized

It is an iterative OpMode: the SDK calls `loop()` about 50 times a second, and each call does a little of the current step. Nothing waits or blocks, so timers and safety checks keep running.

- **Settings at the top:** route, hardware names, motor directions, shooting and driving values. These are the only lines you should need to change.
- **`init()`:** finds each device by name, sets motor directions and brake mode, and resets the encoders. It sets up the launcher's velocity control with goBILDA's tuning (PIDF 40, 0, 0, 12.5) and resets the IMU heading. The launcher, windmill and IMU are each wrapped in `try`, so a missing device switches to a fallback instead of crashing. All four motors are loaded as `DcMotorEx` so the telemetry can read their encoder speed; this doesn't change how they drive. Each device's result (found with its settings, or missing with the error) is written to the Events list.
- **`init_loop()`:** reads the alliance button (B = red, X = blue) and shows the route (in mm and encoder ticks), the shooting and driving settings, each device's status, the live heading, the encoder counts and the Motors section for the hand check, and the INIT events.
- **`start()`:** resets the auto clock and sets heading 0 to the way the robot faces now. If the launcher hardware is missing, it skips straight to LEAVE.
- **`loop()`:** the step-by-step program in the flowchart, plus telemetry (see "Reading the Driver Station screen" below).
- **Driving straight (`startDrive` / `driveDone`):** encoders measure the distance with `RUN_TO_POSITION`, set in the required order: target, then mode, then power. Each loop, the IMU heading error × 0.03 is added to one side's speed limit and subtracted from the other's, steering the robot back onto its line. This is the method of the SDK sample `RobotAutoDriveByGyro`.
- **Turning (`startTurn` / `turnDone`):** turn power = heading error × 0.02, kept between 0.08 and 0.4; left wheels get `-power`, right wheels `+power`. Headings are measured from the start, so a small error in one turn is fixed by the next.
- **Mirroring (`heading()`):** on blue, every heading changes sign. Distances don't change.
- **Safety:** every drive and turn has a timeout. Shooting must end by 17 s so parking always gets time. A turn that moves 30° further from its target stops the robot and shows "Turned the WRONG WAY". A launcher encoder reading about 0 switches the launcher to plain 0.85 power. A motor that turns against its command is reported as `WRONG WAY!`; this only reports, it doesn't stop the robot.

What it needs in the Control Hub configuration

Only the two drive motors are required; every other device has a fallback, but shooting needs all three shooter devices.

Name	Configuration type	Needed?	If it's missing or wrong
`left_drive`	Motor (goBILDA 5203, 312 RPM)	Yes, with its encoder cable plugged in	INIT crashes. Without the encoder, the drive steps time out.

Name	Configuration type	Needed?	If it's missing or wrong
`right_drive`	Motor (goBILDA 5203, 312 RPM)	Yes, with its encoder cable plugged in	Same as `left_drive`
`intake`	Motor	For shooting	If this or `launcher` is missing: no shooting, only LEAVE and PARK
`launcher`	Motor (6000 RPM), encoder plugged in	For shooting	Missing: no shooting. No encoder: plain power, so shot speed varies with the battery.
`windmill`	Continuous Rotation Servo	For shooting well	Feeds with the intake only, and a warning shows on screen
`imu`	Control Hub built-in IMU, added automatically when you scan	Strongly recommended	Falls back to encoder turns, which are less accurate

- **Ports:** any port works as long as the names match. To match the simulator's Ultron Bot, use motor ports 0 (left), 1 (right), 2 (intake) and 3 (shooter).
- **Not used:** `left_intake_servo` and `right_intake_servo`. They can stay in the configuration.
- **Motor directions:** left FORWARD and right REVERSE, as in goBILDA's code and StarterBotAuto. With these, the route values are negative, and negative drives the robot toward the hive (the shooter end).
- **Windmill direction:** REVERSE, so `setPower(1)` feeds the balls, as in goBILDA's code.
- **Intake and launcher directions:** both REVERSE, the same as ShooterAutoTest, ShooterTest and HardwareTest (the shooter tests have been verified on the robot). Without these, the launcher spins backward and the intake pushes balls the wrong way.
- **Hub mounting:** `HUB_LOGO = UP` and `HUB_USB = FORWARD` assume the hub lies flat, logo up. If yours is mounted differently, change these two lines; the hand check catches a wrong setting.

The INIT hand check

After INIT and before START, two hand movements confirm the motor directions, the encoders and the IMU. Here "forward" means away from the hive, the opposite way to how the route drives.

Do this	You should see	If not
Lift the robot and spin both wheels forward by hand	Both counts go up	Counts go down: flip the motor directions. A count stays at 0: check that encoder's cable.
Put it down and turn the robot counter-clockwise by hand	Heading goes up, left count goes down, right count goes up	Left goes up: the motors are swapped in the configuration. Heading goes down: fix `HUB_LOGO` / `HUB_USB`.

While you push, the Motors section shows the same thing as words: wheels forward should read `turning FWD` on both drive lines, with the status `coasting / pushed` (the code isn't driving them, you are).

Reading the Driver Station screen

The screen has three parts, top to bottom. It updates every loop from INIT to DONE.

1. Live values for the current step

Always shown: the state, the alliance, the time spent in this step, the auto clock out of 30 s, and the loop time in ms. Under about 50 ms is fine. Below the step's values, the old lines stay: launcher speed, fed seconds, heading / target, encoder positions and targets, and any WARNING.

Step	What it shows
`SPIN_UP`	Launcher speed against the target (1250) and the ready level (1200), the steady-at-speed time out of 0.3 s, and the spin-up time out of 4 s. With no launcher encoder: the plain-power countdown instead.
`SHOOT`	Feeding ON or PAUSED (launcher below 1200), windmill and intake power, fed seconds out of 4.0 with an estimate of balls fired, the number of feed pauses, and the shoot time against the 10 s and 17 s limits.
`LEAVE`, `TO_PARK`	The distance asked for and its direction, mm driven and mm left for each wheel, the heading error and the steering correction it causes, left / right power, whether the wheels are on target (and for how long, out of 0.25 s), and the time against the timeout.
`TURN_TO_PARK`, `FINAL_TURN`	Turn error (and the error at the start of the turn), turn power for each side, the on-target hold time, and the time against the 3 s timeout. Without an IMU: the ticks left for each wheel.
`DONE`	The auto time when it finished.

2. Motors

One line per motor: `set | told | turning | status`.

- **set:** the direction setting read from the motor (FORWARD or REVERSE).
- **told:** which way the code is telling it to turn right now: FWD, REV or stop. The drive motors' power is always positive while they drive to a target, so for them "told" comes from which side of the target they are on.
- **turning:** which way the encoder says it really turns, with its speed in ticks per second. The SDK applies the direction setting to the encoder too, so "told" and "turning" should always match.

Status	Meaning
`OK`	Turning the way it's told, or stopped when told to stop

Status	Meaning
`starting` / `reversing`	Short, normal moments: speeding up, or overshooting a target
`coasting / pushed`	Told to stop but still moving: the launcher slowing down, or you pushing the robot in INIT
`WRONG WAY!`	Turning against the command for 0.3 s or more. Check that motor's direction setting and its motor and encoder ports. Also written to Events and shown as a WARNING.
`NOT TURNING (no encoder? stuck?)`	Told to turn, but no encoder movement for 1 s. Check the encoder cable, or whether the robot is stuck. Written to Events once. The intake shows this if its encoder isn't plugged in, which is harmless.

The windmill is a servo with no encoder, so its line only shows its setting and the power it's told.

The limits are settings in the Telemetry section at the top of the code: `DIRECTION_MIN_VELOCITY` (50 ticks/s counts as moving), `WRONG_WAY_SECONDS` (0.3) and `NOT_TURNING_SECONDS` (1.0).

3. Events

One line each time a method changes something, stamped with the auto time (or `INIT`). While running, the newest 12 show; at DONE the whole run shows (up to 40), so you can read the full story after the match.

Event	Values shown
INIT, per device	Found with its directions, PIDF and hub mounting, or missing with the error
`START`	Alliance and the starting encoder counts
Step change	From → to, and how long the old step took
Launcher ready / never ready / no encoder	Speed and time taken
First ball fed	Windmill and intake power
`SHOOT` done	Why (all fed, 10 s limit, or 17 s deadline), fed seconds, time taken, number of feed pauses
Launcher off	Velocity 0 or power 0
Drive start	mm, ticks, the targets, the heading held, the timeout
Drive arrived / timed out	mm each wheel drove, the final heading
Turn start	From → to heading, LEFT or RIGHT, the starting error (or the turn arc in ticks without an IMU)
Turn done / timed out / wrong way	Final heading and error
`FINAL_TURN` skipped, `DONE`	The heading and encoder counts at the end
Motor `WRONG WAY` / `NOT TURNING`	Which motor, what it was told, what the encoder read

Before the first field run

The route was measured on the field from the blue base. BioBuzzStarterbotAuto.java now holds these values, written in red form and mirrored for blue.

Setting	Value in the code	Measured
`LEAVE_MM`	-2540	8 ft 4 in, toward and under the hive
`PARK_HEADING_DEG`	-90	Turn left on blue (right on red)
`TO_PARK_MM`	-914	3 ft, on to the blue tape

Fine-tune the route with the Test runs sheet in `TeamCode/docs/BioBuzzStarterbotAuto_Route.xlsx`.

- ☐ Test the `windmill` servo in the TestHardware utility; it has never been tested.
- ☐ BLUE is the default alliance (the INIT screen shows it). Press gamepad1 B for RED, which turns right.
- ☐ Run the route three times and log each miss in the Test runs sheet. After each run, read the Events list at DONE and write down any timeout or warning.
- ☐ During the first run, check that every Motors line says `OK` (the intake may say `NOT TURNING` if its encoder isn't plugged in).
- ☐ Check in the game manual that blue is a mirror of red; if not, set `MIRROR_FOR_BLUE = false`.
- ☐ Copy BioBuzzStarterbotAuto.java into the simulator folder, and delete the old UltronAuto and V2 files there so only one auto shows.